

- [Enterprise RAG Chatbot — Operations & Usage Guide](#)
 - [Table of Contents](#)
 - [1. Project Overview](#)
 - [What This Project Does](#)
 - [Key Capabilities](#)
 - [What Problems It Solves](#)
 - [2. Architecture Deep-Dive](#)
 - [Component Map](#)
 - [Data Flow — Document Ingestion](#)
 - [Data Flow — Query \(RAG\)](#)
 - [Data Flow — Query \(CAG\)](#)
 - [3. Prerequisites](#)
 - [AWS Account Requirements](#)
 - [Local Tools](#)
 - [Enable Bedrock Models](#)
 - [4. Step-by-Step Deployment](#)
 - [Step 1: Clone the Repository](#)
 - [Step 2: Create Terraform State Backend](#)
 - [Step 3: Initialize Terraform](#)
 - [Step 4: Review the Plan](#)
 - [Step 5: Deploy](#)
 - [Step 6: Note the Outputs](#)
 - [Step 7: Verify Health](#)
 - [5. Document Ingestion Guide](#)
 - [Supported File Formats](#)
 - [How to Upload Documents](#)
 - [What Happens After Upload](#)
 - [Best Practices for Documents](#)
 - [Re-indexing a Document](#)
 - [Deleting a Document from the Index](#)
 - [6. Querying the Chatbot](#)
 - [API Endpoint](#)
 - [Request Format](#)
 - [Response Format](#)
 - [Usage Examples](#)
 - [7. Generation Strategies Explained](#)
 - [RAG \(Retrieval-Augmented Generation\) — Default](#)

- [CAG \(Cache-Augmented Generation\)](#)
- [When to Use Which](#)
- [8. Security Controls](#)
 - [Bedrock Guardrails \(Terraform-managed\)](#)
 - [Network Security](#)
 - [Data Encryption](#)
- [9. Monitoring & Troubleshooting](#)
 - [CloudWatch Log Groups](#)
 - [Useful Log Queries](#)
 - [Common Issues & Fixes](#)
- [10. Cost Management](#)
 - [Monthly Cost Breakdown \(Dev Environment\)](#)
 - [Cost Optimization Tips](#)
 - [Teardown \(Stop All Costs\)](#)
- [11. Operational Runbooks](#)
 - [Runbook 1: Add New Documents to Knowledge Base](#)
 - [Runbook 2: Update Lambda Code](#)
 - [Runbook 3: Scale for Production](#)
 - [Runbook 4: Disaster Recovery](#)
- [12. FAQ](#)

Enterprise RAG Chatbot — Operations & Usage Guide

Project: Enterprise RAG Chatbot on Amazon Bedrock

Author: Pushparaj Naik

Version: 1.0

Last Updated: May 2026

Table of Contents

- [1. Project Overview](#)
- [2. Architecture Deep-Dive](#)
- [3. Prerequisites](#)
- [4. Step-by-Step Deployment](#)

- 5. [Document Ingestion Guide](#)
- 6. [Querying the Chatbot](#)
- 7. [Generation Strategies Explained](#)
- 8. [Security Controls](#)
- 9. [Monitoring & Troubleshooting](#)
- 10. [Cost Management](#)
- 11. [Operational Runbooks](#)
- 12. [FAQ](#)

1. Project Overview

What This Project Does

This project deploys an **AI-powered chatbot** that answers questions by searching through your uploaded documents. It uses Amazon Bedrock's Claude model to generate accurate, source-cited answers grounded in your knowledge base — not from the model's pre-training data.

Key Capabilities

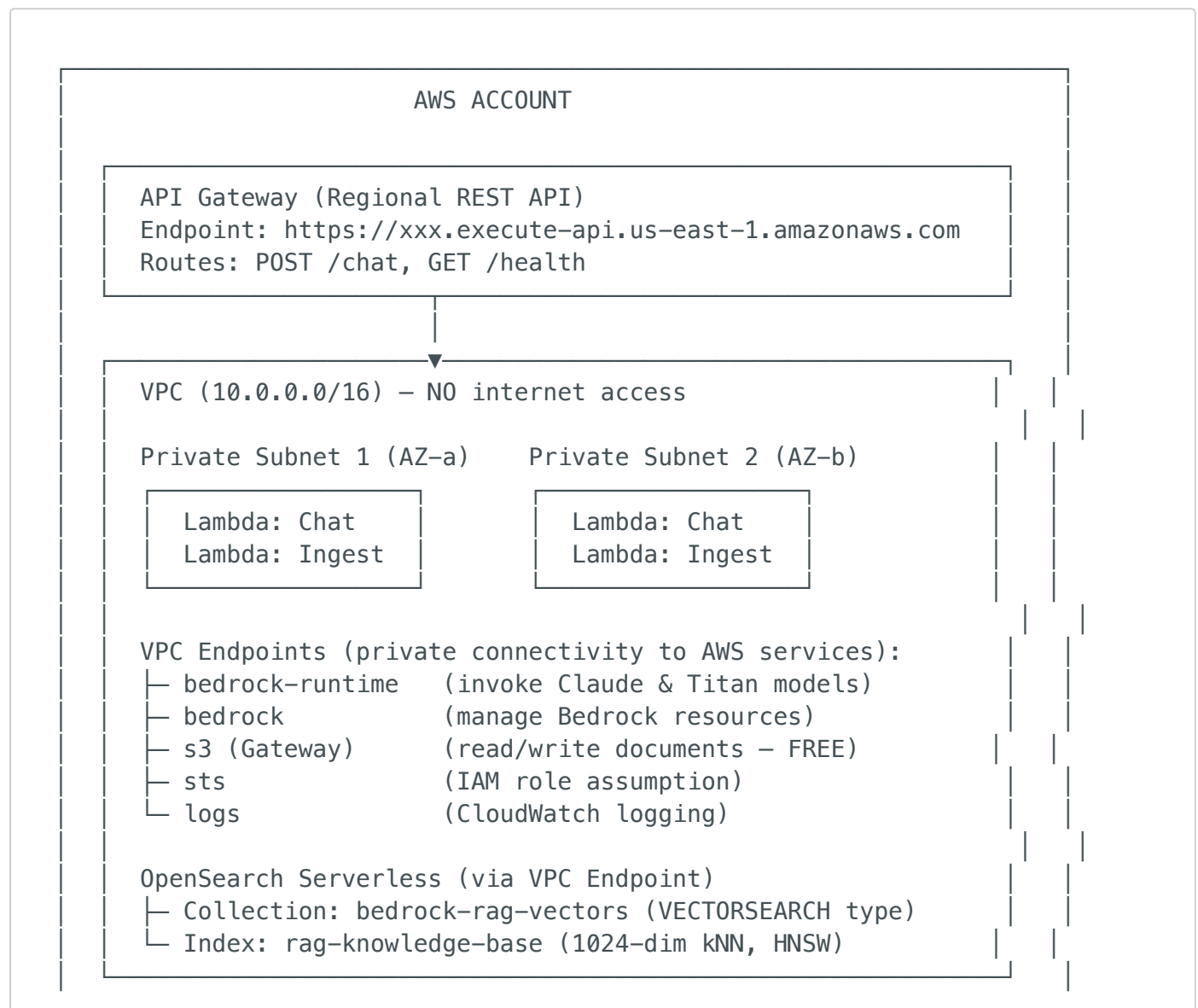
Capability	Description
Document Ingestion	Upload PDF, TXT, or MD files → auto-parsed, chunked, embedded, and indexed
RAG Querying	Ask questions → system finds relevant chunks via vector search → generates grounded answer
CAG Querying	For small knowledge bases — loads full KB into prompt with caching for 90% cost savings
Content Safety	Bedrock Guardrails filter PII, harmful content, prompt attacks, and off-topic queries
Enterprise Security	Private VPC, KMS encryption, IAM least-privilege, no internet egress

What Problems It Solves

- **"Our team wastes hours searching for information across documents"** → Upload all docs, ask natural language questions
 - **"We can't use public ChatGPT due to data privacy"** → Fully private, data never leaves your AWS account
 - **"LLMs hallucinate and make things up"** → RAG grounds answers in your actual documents with source citations
 - **"AI costs are unpredictable"** → CAG prompt caching reduces repeated query costs by 90%
-

2. Architecture Deep-Dive

Component Map



S3 Bucket
/documents/ (raw)
/processed/ (txt)
/embeddings/
/logs/

Bedrock
Claude 3.5 Sonnet
Titan Embed v2
Guardrails
(PII, Content)

KMS CMK
Auto-rotation: ON

Encrypts: S3, Lambda env vars,
CloudWatch Logs

Data Flow — Document Ingestion

Step 1: User uploads file

```
aws s3 cp policy.pdf s3://bedrock-rag-documents-dev/documents/
```

Step 2: S3 sends event notification to Ingest Lambda

Event: s3:ObjectCreated:* on prefix "documents/"

Step 3: Ingest Lambda processes the document

3a. Read document from S3

3b. Parse text content (UTF-8)

3c. Chunk into ~512 character segments with 50 char overlap
→ Semantic boundaries: paragraphs first, then sentences

3d. For each chunk:

→ Call Titan Embed v2 → generate 1024-dim vector

→ Index into OpenSearch: { text, embedding, source_file, metadata }

3e. Save full processed text to s3://bucket/processed/ (for CAG)

Step 4: Document is searchable immediately

Data Flow — Query (RAG)

Step 1: User sends query

```
POST /chat {"query": "What is the refund policy?", "strategy": "rag"}
```

Step 2: API Gateway → Chat Lambda

Step 3: RAG Engine executes:

3a. Embed the query using Titan Embed v2 → 1024-dim vector

3b. kNN search on OpenSearch → retrieve top-5 most similar chunks

3c. Build prompt:

"Here is the context: [chunk1] [chunk2] ... Answer this: {query}"

- 3d. Invoke Bedrock Claude 3.5 Sonnet with the prompt
- 3e. Claude generates answer grounded in the chunks

Step 4: Return response with answer + sources + cost estimate

Data Flow — Query (CAG)

Step 1: User sends query

```
POST /chat {"query": "What is the refund policy?", "strategy": "cag"}
```

Step 2: API Gateway → Chat Lambda

Step 3: CAG Engine executes:

- 3a. Load ALL processed documents from S3 (cached in Lambda memory)
- 3b. Build prompt with ENTIRE knowledge base + cache_control flag
- 3c. Invoke Bedrock Claude with prompt caching enabled
- 3d. First query: cache_creation (25% premium on KB tokens)
Subsequent queries: cache_read (90% CHEAPER on KB tokens)

Step 4: Return response with answer + cache hit status + cost estimate

3. Prerequisites

AWS Account Requirements

Requirement	How to Verify
AWS Account with admin access	<code>aws sts get-caller-identity</code>
Bedrock model access enabled	AWS Console → Bedrock → Model access → Enable Claude 3.5 Sonnet + Titan Embed v2
Region: us-east-1	Models must be available in your region
S3 bucket for Terraform state	Create: <code>bedrock-rag-terraform-state</code>
DynamoDB table for state lock	Create: <code>terraform-lock</code> (Partition key: <code>LockID</code> , type String)

Local Tools

```
# Terraform >= 1.5
terraform --version

# AWS CLI v2
aws --version

# Python >= 3.12
python3 --version

# Configure AWS credentials
aws configure
# Or use SSO: aws sso login --profile your-profile
```

Enable Bedrock Models

1. Go to AWS Console → Amazon Bedrock → Model access
2. Click "Manage model access"
3. Enable:
 - ✓ Anthropic → Claude 3.5 Sonnet v2
 - ✓ Amazon → Titan Text Embeddings V2
4. Wait for access to be granted (usually instant)

4. Step-by-Step Deployment

Step 1: Clone the Repository

```
cd /Users/pushparajnaik/Desktop/Pushparaj\
Naik/TerraformCode/NewProjects/Bedrock_RAG
```

Step 2: Create Terraform State Backend

```
# Create S3 bucket for state (one-time setup)
aws s3mb s3://bedrock-rag-terraform-state --region us-east-1
```

```
# Create DynamoDB lock table (one-time setup)
aws dynamodb create-table \
  --table-name terraform-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

Step 3: Initialize Terraform

```
cd terraform
terraform init
```

Expected output:

```
Terraform has been successfully initialized!
```

Step 4: Review the Plan

```
terraform plan
```

Review the output. You should see ~25 resources being created:

- 1 VPC, 2 Subnets, 3 Security Groups
- 5 VPC Endpoints
- 1 S3 Bucket (with encryption, versioning, lifecycle)
- 1 OpenSearch Serverless Collection + VPC Endpoint + policies
- 1 IAM Role with 5 scoped policies
- 2 Lambda Functions (chat + ingest)
- 1 API Gateway with 3 endpoints
- 1 Bedrock Guardrail
- 1 KMS Key

Step 5: Deploy

```
terraform apply
```

Type **yes** when prompted. Deployment takes **5-10 minutes** (OpenSearch Serverless collection creation is the slowest).

Step 6: Note the Outputs

```
terraform output
```

```
api_gateway_url    = "https://abc123.execute-api.us-east-1.amazonaws.com/dev"
s3_bucket_name     = "bedrock-rag-documents-dev"
opensearch_endpoint = "https://xyz.us-east-1.aoss.amazonaws.com"
```

Step 7: Verify Health

```
API_URL=$(terraform output -raw api_gateway_url)
curl -s "$API_URL/health" | python3 -m json.tool
```

Expected:

```
{
  "status": "healthy",
  "service": "enterprise-rag-chatbot",
  "timestamp": "2026-05-07T16:30:00Z"
}
```

5. Document Ingestion Guide

Supported File Formats

Format	Extension	Notes
Plain text	<code>.txt</code>	Best for structured content
Markdown	<code>.md</code>	Preserves headers and formatting
PDF	<code>.pdf</code>	Basic text extraction (not scanned images)

How to Upload Documents

```
# Single file
aws s3 cp company-policy.txt s3://bedrock-rag-documents-dev/documents/

# Multiple files
aws s3 cp ./knowledge-base/ s3://bedrock-rag-documents-dev/documents/ --recursive

# Verify upload
aws s3 ls s3://bedrock-rag-documents-dev/documents/
```

What Happens After Upload

Upload triggers S3 event → Ingest Lambda runs automatically

Monitor in CloudWatch:

```
aws logs tail /aws/lambda/bedrock-rag-ingest --follow
```

You'll see logs like:

```
Processing: s3://bedrock-rag-documents-dev/documents/company-policy.txt
Read document: 15234 chars
Text chunked into 12 chunks (avg 480 chars)
Generated 12 embeddings
Indexed 12/12 chunks for company-policy.txt
```

Best Practices for Documents

Practice	Why
Use clear headings	Improves chunking quality
Keep one topic per document	Better retrieval accuracy
Remove boilerplate (headers, footers)	Reduces noise in search results
Use UTF-8 encoding	Prevents character encoding issues
Max 10MB per file	Lambda memory constraint
Upload 5-10 docs at a time	Avoid Lambda throttling

Re-indexing a Document

```
# Simply re-upload – the indexer overwrites existing chunks by document ID
aws s3 cp updated-policy.txt s3://bedrock-rag-documents-dev/documents/
```

Deleting a Document from the Index

Currently requires manual OpenSearch API call:

```
# Delete all chunks for a specific document
curl -X POST "$OPENSEARCH_ENDPOINT/rag-knowledge-base/_delete_by_query" \
  -H "Content-Type: application/json" \
  -d '{"query": {"term": {"source_file": "old-policy.txt"}}}'
```

6. Querying the Chatbot

API Endpoint

```
POST https://<api-gateway-url>/chat
Content-Type: application/json
```

Request Format

```
{
  "query": "What is our return policy for electronics?",
  "strategy": "rag",
  "top_k": 5
}
```

Field	Type	Required	Default	Description
query	string	☒ Yes	—	Your question
strategy	string	No	"rag"	"rag" or "cag"
top_k	integer	No	5	Number of context chunks (RAG only, max 20)

Response Format

```
{
  "answer": "Based on the company policies document, electronics can be returned within 30 days of purchase with original receipt...",
  "sources": ["company-policies.txt", "return-guide.md"],
  "strategy": "rag",
  "model": "anthropic.claude-3-5-sonnet-20241022-v2:0",
  "latency_ms": 1842,
  "token_usage": {
    "input_tokens": 1523,
    "output_tokens": 312,
    "estimated_cost_usd": 0.009249
  }
}
```

Usage Examples

```
API_URL="https://abc123.execute-api.us-east-1.amazonaws.com/dev"

# Example 1: RAG query (default – best for large knowledge bases)
curl -s -X POST "$API_URL/chat" \
  -H "Content-Type: application/json" \
  -d '{"query": "What are the vacation day policies?"}' | python3 -m json.tool
```

```
# Example 2: RAG with more context chunks
curl -s -X POST "$API_URL/chat" \
  -H "Content-Type: application/json" \
  -d '{"query": "Explain the security incident response procedure", "top_k": 10}' | python3 -m json.tool

# Example 3: CAG query (cheaper for repeated queries on small KB)
curl -s -X POST "$API_URL/chat" \
  -H "Content-Type: application/json" \
  -d '{"query": "What are the vacation day policies?", "strategy": "cag"}' | python3 -m json.tool
```

7. Generation Strategies Explained

RAG (Retrieval-Augmented Generation) — Default

Query → Embed → Vector Search (top-K) → Build Prompt → LLM → Answer

Pros	Cons
Works with any KB size (1000+ pages)	Higher latency (embed + search + LLM)
Only sends relevant chunks to LLM	May miss context if chunks aren't well-matched
Cost scales with query count, not KB size	Requires OpenSearch Serverless (min \$30/month)

Use when: Knowledge base is large, diverse, or frequently updated.

CAG (Cache-Augmented Generation)

Load Full KB → Cache in Prompt → LLM (with prompt caching) → Answer

Pros	Cons
Full context available — no retrieval gaps	KB must fit in context window (~200K tokens)
90% cheaper on repeated queries (cache hits)	First query is 25% more expensive (cache creation)
Simpler architecture (no vector search)	Not suitable for large KB (>100 pages)
Lower latency on cache hits	Cache expires after 5 minutes of inactivity

Use when: Knowledge base is small (<100 pages) and queried frequently.

When to Use Which

Scenario	Recommended
500-page employee handbook	RAG
10-page FAQ document	CAG
Frequently asked same questions	CAG (cache hits)
Knowledge base changes daily	RAG
Maximum answer accuracy needed	RAG with top_k=10
Cost is the primary concern	CAG for small KB, RAG for large KB

8. Security Controls

Bedrock Guardrails (Terraform-managed)

Control	Action	What It Catches
PII — Email	Anonymize	user@company.com → [EMAIL]
PII — Phone	Anonymize	555-0123 → [PHONE]
PII — SSN	Block	123-45-6789 → Request blocked entirely

Control	Action	What It Catches
PII — Credit Card	Block	4111-1111-1111-1111 → Request blocked
Sexual content	Block (HIGH)	Explicit content in query or response
Violence	Block (HIGH)	Violent content
Hate speech	Block (HIGH)	Discriminatory content
Insults	Block (HIGH)	Personal attacks
Misconduct	Block (HIGH)	Illegal activities
Prompt attacks	Block (HIGH)	Jailbreak attempts, prompt injection
Topic: Competitors	Deny	"What does competitor X charge?"
Topic: Internal financials	Deny	"What is the company revenue?"

Network Security

- **No internet egress:** Lambda runs in private subnets with NO NAT Gateway
- **VPC Endpoints only:** All AWS API calls go through PrivateLink (within AWS backbone)
- **Security Groups:** Lambda SG → outbound only. OpenSearch SG → inbound from Lambda SG on 443 only

Data Encryption

Data	Encryption	Key
S3 documents	SSE-KMS	Project CMK (auto-rotated annually)
Lambda env vars	KMS	Project CMK

Data	Encryption	Key
CloudWatch Logs	KMS	Project CMK
OpenSearch data	AWS-owned key	Managed by AWS
API Gateway traffic	TLS 1.2+	AWS-managed cert
VPC Endpoint traffic	TLS 1.2+	AWS-managed

9. Monitoring & Troubleshooting

CloudWatch Log Groups

Log Group	What It Contains
/aws/lambda/bedrock-rag-chat	Query processing, Bedrock invocations, errors
/aws/lambda/bedrock-rag-ingest	Document parsing, chunking, embedding, indexing
/aws/apigateway/bedrock-rag	API request/response logs

Useful Log Queries

```
# Watch chat Lambda logs in real-time
aws logs tail /aws/lambda/bedrock-rag-chat --follow

# Watch ingest Lambda logs
aws logs tail /aws/lambda/bedrock-rag-ingest --follow

# Search for errors in last 1 hour
aws logs filter-log-events \
  --log-group-name /aws/lambda/bedrock-rag-chat \
  --start-time $(date -v-1H +%s000) \
  --filter-pattern "ERROR"
```

Common Issues & Fixes

Symptom	Cause	Fix
502 Bad Gateway	Lambda timeout (>29s API GW limit)	Increase Lambda memory or reduce top_k
No relevant information found	Documents not indexed	Check ingest Lambda logs, verify S3 prefix is documents/
Connection refused (OpenSearch)	VPC endpoint not ready	Wait 5 min after deployment, check SG rules
AccessDeniedException (Bedrock)	Model not enabled	Enable Claude + Titan in Bedrock Console → Model access
High latency (>5s)	Lambda cold start	Increase memory to 1024MB+, or enable provisioned concurrency
ResourceNotFoundException (index)	Index not created yet	Upload a document first — ingest Lambda creates the index

10. Cost Management

Monthly Cost Breakdown (Dev Environment)

Service	Cost	Notes
OpenSearch Serverless	~\$30	2 OCU minimum (indexing + search)
VPC Endpoints (3 Interface)	~\$21	\$7/month each for Bedrock, STS, Logs
Lambda	~\$2-5	Pay per invocation (1M free tier)
API Gateway	~\$1-3	\$1 per million requests
S3	< \$1	Storage + requests

Service	Cost	Notes
KMS	~\$1	1/ <i>key/month</i> +0.03/10K requests
Bedrock (Claude)	Variable	~0.003/1K <i>inputtokens</i> ,0.015/1K output
Bedrock (Titan Embed)	Variable	~\$0.0002/1K tokens
Total (base)	~\$55-60/month	

Cost Optimization Tips

1. Use CAG for frequently queried small KBs → 90% savings on Bedrock tokens
2. Set top_k=3 instead of 5 → fewer input tokens per query
3. Use temperature=0.1 → shorter, more focused responses (fewer output tokens)
4. Disable OpenSearch standby replicas in non-prod (already done in Terraform)
5. Set CloudWatch log retention to 7 days in dev (currently 30)
6. Delete non-production environments after hours

Teardown (Stop All Costs)

```
cd terraform
terraform destroy
```

11. Operational Runbooks

Runbook 1: Add New Documents to Knowledge Base

```
# 1. Prepare documents (UTF-8 text, <10MB each)
# 2. Upload to S3
aws s3cp new-document.txt s3://bedrock-rag-documents-dev/documents/
```

Runbook 2: Update Lambda Code

Runbook 3: Scale for Production

Runbook 4: Disaster Recovery

```
# Terraform state is in S3 (versioned) – recoverable
# OpenSearch data can be rebuilt by re-ingesting documents from S3
# S3 bucket has versioning enabled – accidental deletes recoverable

# Full recovery:
terraform apply                # Recreate all infrastructure
```

```
aws s3 ls s3://bedrock-rag-documents-dev/documents/ # Verify docs exist
# S3 notifications will auto-trigger re-ingestion of all documents
```

12. FAQ

Q: Can I use a different Bedrock model?

A: Yes. Change `bedrock_model_id` in `terraform/variables.tf`. Supported models: Claude 3.5 Sonnet, Claude 3 Haiku (cheaper, faster), Claude 3 Opus (most capable).

Q: Can I use this with private/confidential documents?

A: Yes. Data never leaves your AWS account. Lambda runs in a private VPC with no internet access. All traffic to AWS services goes through VPC Endpoints (PrivateLink).

Q: How many documents can I index?

A: OpenSearch Serverless scales automatically. Tested with 10,000+ documents. For very large KBs, increase the ingest Lambda timeout and memory.

Q: What's the maximum query latency?

A: Typical: 2-4 seconds (RAG), 1-3 seconds (CAG with cache hit). API Gateway timeout is 29 seconds.

Q: Can I add authentication?

A: Yes. The API Gateway module is pre-configured with a `NONE` authorizer. Switch to `COGNITO_USER_POOLS` and add a Cognito User Pool for JWT-based auth.

Q: How do I monitor costs?

A: The API response includes `estimated_cost_usd` per query. Set up AWS Budgets for monthly alerts.

Q: Can I deploy to multiple regions?

A: Yes. Create a separate `terraform/environments/us-west-2/` with the same modules. Bedrock model availability varies by region.